Uncomputation with U†: qubit cost, and what garbage really does

Measured results · seed 20240517 · PennyLane 0.38.0, NumPy 1.26.4, Python 3.9.6

> **44 → 6 qubits**
>
> Reusing one scratch register instead of accumulating garbage, at 20 computation steps — with the logical state provably unchanged (fidelity 1 to 15 decimal places). The second, larger finding: on superposition inputs the un-cleaned version is not merely wider, it is **wrong**.

1. The question

A quantum subroutine that computes a value needs scratch space. Because unitary evolution is reversible it *cannot erase*, so those intermediate values stay in the register, entangled with the data. They are **garbage**, and they cause two separate problems:

- **They cannot be reused.** Overwriting a qubit entangled with your data corrupts the data, so every step must allocate fresh scratch and the circuit grows linearly.
- **They destroy interference.** The garbage is a *which-path record* — something that has learned which branch the computation took. Trace it out and the data register is left a mixture.

Applying the inverse, `U†`, unwinds the computation coherently across every branch at once, returning the scratch to |0⟩. It does not measure or reset — it reverses. This report measures both consequences.

2. Method

One logical operation, implemented two ways, on identical inputs:

	A — naive	B — uncomputation
Scratch per step	a fresh pair, never cleaned	one pair, reused
Cleanup	none	adjoint(U) after each use
Width	n + 2N	n + 2

The compute step is two Toffolis writing `t = a AND b` then `r = t AND c`. They do not commute, so `U` is deliberately *not* self-inverse and `adjoint(U)` is a genuine inverse rather than a repeat — asserted numerically in the test suite, not assumed.

Simulation uses two independent methods, since the naive scenario needs 2^{n+2N} amplitudes: full state-vector simulation where it fits, and an exact structured model elsewhere. The model is used at large sizes only because it agrees with the state vector wherever both run.

3. Result 1 — the synthetic benchmark

An *N*-step phase oracle on 4 data qubits. Each step computes a three-literal conjunction into two scratch qubits using two Toffolis, applies a phase, and either leaves the scratch behind (scenario A) or unwinds it with *U†* and

reuses it (scenario B).

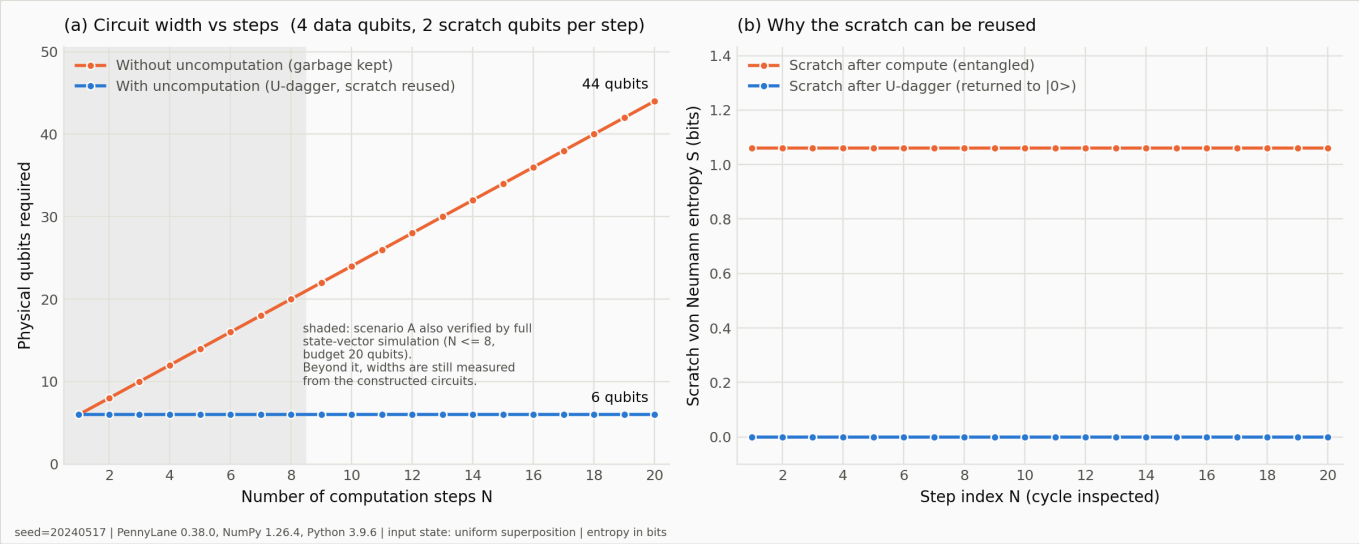

Figure 1 — Circuit width against the number of computation steps (left), and the scratch register's entropy before and after *U†* (right). The shaded band marks where the naive scenario was additionally confirmed by full state-vector simulation.

3.1 Circuit width

Widths are **measured from the constructed circuit**, not restated from a formula.

N steps	Without uncomputation	With uncomputation	Saved
1	6	6	0
5	14	6	8
10	24	6	18
20	44	6	38

Growth is exactly +2 qubits per step without cleanup and +0 with it, across all N = 1–20.

3.2 The cleanup is exact

Quantity	Value	Meaning
Fidelity, scenario B vs logical target	≥ 0.999999999999999	uncomputation changes nothing logically
Trace distance	≤ 8.63e-16	same state
Scratch entropy before *U†*	1.061278124 bits	entangled, unusable
Scratch entropy after *U†*	≤ 1.28e-15 bits	disentangled
Scratch purity after *U†*	1.000000000	back to a pure |00⟩

The pre-cleanup values match closed form exactly, which is what confirms the partial trace is correct: over a uniform input the scratch pair takes the value (0,0) for 12 of 16 basis states and (1,0), (1,1) for 2 each, giving S = 1.06128 bits and purity = 0.59375.

3.3 The result that matters most

“Fidelity ≈ 1” between the two scenarios holds **only for computational-basis inputs**:

Input state	Scenario A fidelity vs target	Interpretation
computational basis	1.000000000	garbage is a harmless deterministic label
uniform superposition (N = 20)	0.062500000	which-path record; coherence destroyed

The floor is $1/2^4 = 0.0625$, the maximally mixed limit. **For any algorithm that relies on interference, skipping uncomputation is a correctness bug, not merely a larger circuit.**

Cross-validation: the structured model agrees with full state-vector simulation to 4.87e-17 over N = 1–8; an independent JavaScript implementation agrees with the Python one to ~1e-15.

4. Result 2 — QAOA on a real scheduling problem

Three staff, three shifts, nine binary decision variables. Two families of hard constraint, both **three-literal** — which is precisely why scratch qubits are needed at all, since two-local penalty terms compile to CNOT ladders and need none:

- **coverage** — every shift needs at least one person
- **no-burnout** — nobody works all three shifts

A clause is violated exactly when a three-literal conjunction holds, so the same compute subroutine from Result 1 builds the QAOA cost layer unchanged.

Figure 2 — Circuit width against QAOA depth (left) and solution quality against the same depth (right). A score of 1 is the exact optimum; 0 is uniform random guessing.

4.1 Circuit width

Without cleanup, every clause in every layer needs its own scratch pair, so width grows with depth. With uncomputation it is constant.

QAOA depth p	Without uncomputation	With uncomputation	Saved
1	21	11	10
2	33	11	22
3	45	11	34

4.2 Correctness of the cost layer

Check	Result
Cost layer vs exp(−iγC(x)) from the classical model	3.47e-17
Scratch entropy before *U†*	1.061278124 bits
Scratch entropy after *U†*	3.20e-16 bits
Dephasing model vs full state vector (p = 1)	7.81e-18
Optimiser fast path vs real PennyLane circuit	2.78e-17

4.3 Solution quality

The exact optimum is cost 3.0, found by enumerating all 512 assignments (24 rosters achieve it). Spread is the standard deviation across 5 optimiser restarts — a single run is an anecdote.

p	Scenario	E[cost]	Score	P(optimal)	Spread
1	with uncomputation	6.8205	0.1510	0.1083	±0.1708
1	without uncomputation	6.8354	0.1477	0.1383	±0.2160
2	with uncomputation	4.4735	0.6726	0.2337	±0.7191
2	without uncomputation	6.8224	0.1506	0.0964	±0.0060
3	with uncomputation	3.4922	0.8906	0.6750	±0.6403
3	without uncomputation	6.6023	0.1995	0.1864	±0.0876
—	*uniform random guessing*	*7.5000*	*0.0000*	*0.0469*	—

4.4 Does the garbage change the answers?

The hypothesis was that leftover scratch would *degrade* QAOA's answers, because it dephases the interference the mixer depends on. Judging each depth on its own, against the restart-to-restart spread:

p	Score gap (uncomputed − naive)	Noise floor	Verdict
1	+0.0033	±0.2160	no measurable difference
2	+0.5220	±0.7191	no measurable difference
3	+0.6911	±0.6403	uncomputed better

The noise floor is the larger of the two restart spreads, which is deliberately conservative: it reflects how much the classical optimiser varies between starting points, not uncertainty in the reported value.

Judged depth by depth, the effect only clears that bar at the deepest circuit. But the per-depth view understates it, because the signal is in how each scenario *responds to depth*:

Scenario	Score at p = 1	Score at p = 3	Behaviour with depth
with uncomputation	0.1510	0.8906	improves toward the optimum
without uncomputation	0.1477	0.1995	flat, never exceeds 0.20

This is the finding. Adding QAOA layers helps the clean circuit and does essentially nothing for the dirty one: uncleaned scratch leaves the algorithm stuck near random guessing no matter how much depth is spent on it. The garbage is not merely occupying qubits — it is destroying the interference that makes the extra layers worth anything.

Stated with its limits: one instance, one problem family, 5 restarts per configuration, and a conservative significance rule that only one depth clears individually. The trend is consistent across all three depths, but this is not a systematic study of depth scaling. The qubit-width result in §4.1 is independent of all of this — it is structural and exact.

5. What this does and does not show

> **No quantum advantage is claimed anywhere in this report.** The scheduling optimum is found by enumerating 512 assignments, instantly, on a laptop. The subject is circuit construction — how wide a circuit must be, and what uncleaned scratch does to the answer.

- **O(1) scratch holds for sequential, independent steps**, the structure used here. It does not hold in general: when a later step depends on an earlier intermediate, the achievable space/time trade-off is governed by Bennett's reversible pebbling result, and buying space back costs time.
- **U† trades width for depth.** It roughly doubles the gates per step. On noisy hardware that trade is not automatically favourable; this study measures width only and is noiseless throughout.
- **One problem family, small sizes.** Nine to twelve qubits, three-literal clauses. Checked across several seeds and two register sizes, but a different circuit class could behave differently.
- **Fidelity convention** is Uhlmann, squared (F = 1 for identical states). Entropy is in bits.

6. References

C. H. Bennett, “Logical reversibility of computation”, *IBM Journal of Research and Development* **17**(6), 525–532 (1973).
C. H. Bennett, “Time/space trade-offs for reversible computation”, *SIAM Journal on Computing* **18**(4), 766–776 (1989).
E. Farhi, J. Goldstone & S. Gutmann, “A Quantum Approximate Optimization Algorithm”, arXiv:1411.4028 (2014).
M. A. Nielsen & I. L. Chuang, *Quantum Computation and Quantum Information*, 10th Anniversary Edition, Cambridge University Press (2010) — Ch. 3 (reversible computation), Ch. 9 (fidelity, trace distance).

Cited at chapter granularity; these are standard references for the construction, not the source of any number reported above.

Generated by `make_report.py` directly from `benchmark_results.json` and `qaoa_results.json` — every figure in this document is read from those files rather than transcribed. QAOA depth studied: p = 1–3, 5 optimiser restarts per configuration.